CASE 1: Financial Data Summarisation

Tool: Microsoft Copilot
Objective: Summarise complex SQL findings into a senior management executive summary

BUSINESS PROBLEM
FP&A teams regularly need to distil large volumes of data into concise, decision-ready summaries for senior management. This use case tests Copilot's ability to produce accurate, professional executive summaries from raw analytical findings.

PROMPT USED
I have retail sales data from 10 shopping malls in Turkey containing 99,000 transactions. Here are the key findings from my SQL analysis:

- Total revenue across all categories: Clothing led with 113,996,791 TL, followed by Shoes, Technology, and Cosmetics
- Technology had the highest average transaction value at 11,581 TL despite lower transaction volume
- Mall of Istanbul generated the highest revenue at 50,872,481 TL across 19,943 transactions
- Revenue peaks in July and October each year; lowest in February
- Top 10% of transactions (High Value tier) had spent above 26,250 TL
- Mall of Istanbul had the highest concentration of High Value transactions at 2,026

Please summarise these findings into a concise executive summary suitable for senior management, maximum 150 words, in a professional tone, highlighting the 3 most important business insights.

COPILOT OUTPUT
Executive Summary – Retail Sales Analysis (Turkey, 99,000 Transactions)

Our analysis of retail sales across 10 shopping malls highlights three critical insights:

1. Category Performance: Clothing dominates revenue at 113.9M TL, while Technology stands out with the highest average transaction value (11,581 TL) despite lower volume, signaling strong potential in premium product positioning.
2. Mall Leadership: Mall of Istanbul is the clear revenue leader, generating 50.9M TL across nearly 20,000 transactions, and hosting the highest concentration of High Value transactions (2,026). This underscores its role as a strategic hub for affluent customers.
3. Seasonality: Revenue consistently peaks in July and October, with February as the weakest month, suggesting opportunities for targeted promotions to smooth seasonal dips.

These findings emphasise the importance of leveraging Mall of Istanbul's high-value customer base, capitalising on Technology's premium appeal, and optimising seasonal campaigns to maximise overall growth.

Critical Evaluation:

What Copilot did well:

- Structured the output into 3 clearly labelled insights appropriate for senior management consumption
- Correctly identified the Clothing vs Technology contrast as the most important category finding
- Picked up the seasonality pattern accurately and connected it to a business action (promotional campaigns)
- Stayed within the 150-word limit and maintained professional tone throughout

What Copilot missed or got wrong:

- The summary says "strong potential in premium product positioning" for Technology this is a generic recommendation not supported by the data. The data shows Technology has high average spend but we have

no information on margin, growth trend, or customer demographics for that category. A real analyst would not make that leap.
- "Optimizing seasonal campaigns to maximize overall growth" is vague filler. A stronger recommendation would specify which months need campaign support and what categories to target during February specifically.
- Copilot did not flag that Mall of Istanbul leading on both revenue AND high-value transactions could mean over-concentration risk all eggs in one basket. That's a risk insight a senior analyst would raise.

What I corrected or added:

- Added caveat that Technology recommendation requires further demographic and margin data before acting on it
- Specified February as the target month for promotional intervention rather than leaving it as a general seasonal observation
- Flagged Mall of Istanbul concentration as both an opportunity and a dependency risk

Analyst Verdict
Copilot produced a solid first draft in under 10 seconds. It required one round of critical review to remove unsupported recommendations and sharpen the actionability of the output. Useful as a starting point, not as a final deliverable.

Case 2: Variance Commentary Drafting.

Tool: Microsoft Copilot
Objective: Generate professional variance commentary explaining month-on-month and budget variances for senior management reporting

BUSINESS PROBLEM
FP&A analysts spend significant time writing variance commentary for monthly reporting packs. The commentary must be accurate, concise, and explain both the numbers and the business reasons behind them. This use case tests Copilot's ability to produce publication-ready variance commentary from structured data inputs.

PROMPT USED
I am an FP&A analyst preparing monthly variance commentary for senior management. Here is the data: Category: Clothing Current Month Revenue: 9,845,000 TL Prior Month Revenue: 10,311,119 TL Variance: -466,119 TL (-4.5%) Budget: 10,000,000 TL Variance vs Budget: -155,000 TL (-1.6%) Context: February is historically the lowest revenue month. Clothing is the highest revenue category overall. Write a professional variance commentary paragraph explaining these results to senior management include the variance vs prior month, variance vs budget, and a possible explanation for the shortfall.

COPILOT OUTPUT
Clothing revenue for February totalled 9.85M TL, representing a 4.5% decline (-466K TL) versus January and a 1.6% shortfall (-155K TL) against budget. The month-on-month decrease reflects the seasonal dip typically observed in February, historically the lowest revenue month across categories. While Clothing remains the highest revenue contributor overall, performance was slightly below expectations due to softer footfall and discretionary spending during the post-holiday period. The variance versus budget is modest, and given the category's strong annual trajectory, this shortfall is consistent with seasonal patterns rather than structural weakness.

CRITICAL EVALUATION

What Copilot did well:

- Correctly identified and stated both variances vs prior month (-4.5%, -466K TL) and vs budget (-1.6%, -155K TL) in a single coherent paragraph
- Connected the shortfall to seasonal context (February as historically weakest month) without being prompted to do so
- Used appropriate professional language "softer footfall", "discretionary spending", "structural weakness" consistent with real FP&A commentary style
- Correctly framed the budget variance as modest and non-alarming rather than overstating the risk

What Copilot missed or got wrong:

- "Softer footfall and discretionary spending during the post-holiday period" is an assumption the dataset contains no footfall data or information about consumer sentiment. Copilot generated a plausible explanation but presented it as fact rather than a hypothesis
- The commentary does not mention what actions, if any, are being taken to address the shortfall or whether a recovery is expected in March real management commentary typically includes a forward-looking statement
- "Strong annual trajectory" is asserted without referencing any specific data point a finance director would ask what that trajectory actually is

What I corrected or added:

- Flagged that the footfall explanation is an assumption and would need to be verified against operational data before inclusion in a real management pack
- Added a forward-looking line: "Revenue is expected to recover in March as seasonal patterns normalise, consistent with prior year trends showing stronger performance from Month 3 onwards"
- Replaced "strong annual trajectory" with the specific data point: Clothing generated 113.9M TL in annual revenue, ranking #1 across all categories

ANALYST VERDICT

Copilot produced a well-structured first draft in under 10 seconds that would pass an initial review. However, it fabricated a causal explanation (footfall) not supported by the data and omitted a forward-looking statement. Suitable as a drafting tool but requires analyst review before submission to senior management outputs should never be used without verification against actual data.

CASE 3: Data Audit Query Generation

Tool: Microsoft Copilot
Objective: Generate SQL queries to identify data quality issues across a retail sales database

BUSINESS PROBLEM

Data administrators and analysts need to regularly audit large datasets for quality issues before running analysis or making business decisions. Writing audit queries manually is time-consuming. This use case tests Copilot's ability to generate accurate, well-commented SQL audit queries from a plain English brief directly relevant to data integrity roles in operations and systems administration.

PROMPT USED

I am a data analyst performing a data quality audit on a retail sales database. The database has the following columns: invoice_no, customer_id, gender, age, category, quantity, price, payment_method, invoice_date, shopping_mall

I need to check for the following data quality issues:
1. Duplicate invoice numbers
2. Negative or zero values in quantity and price

3. Future-dated transactions
4. Age values outside a realistic range (below 15 or above 90)
5. Transactions where quantity or price is NULL

Write me SQL queries to check for each of these issues, with a brief comment explaining what each query is checking and why it matters for data integrity.

COPILOT OUTPUT

1. Duplicate invoice numbers

```
SELECT invoice_no, COUNT() AS duplicate_count
 FROM retail_sales
GROUP BY invoice_no
HAVING COUNT() > 1;
-- Checks for duplicate invoice numbers, which can indicate data entry errors or system issues affecting transaction uniqueness.
```

2. Negative or zero values in quantity and price

```
SELECT *
FROM retail_sales
WHERE quantity <= 0 OR price <= 0;
-- Identifies invalid transaction values. Quantity and price should always be positive; negative or zero values compromise revenue accuracy.
```

3. Future-dated transactions

```
SELECT *
FROM retail_sales
WHERE invoice_date > CURRENT_DATE;
-- Flags transactions dated in the future, which may result from input errors or incorrect system timestamps.
```

4. Age values outside realistic range (below 15 or above 90)

```
SELECT *
FROM retail_sales
WHERE age < 15 OR age > 90;
-- Ensures customer demographic data is realistic. Out-of-range ages reduce reliability of customer segmentation and analytics.
```

5. Transactions where quantity or price is NULL

```
SELECT *
FROM retail_sales
WHERE quantity IS NULL OR price IS NULL;
-- Detects missing critical values. NULLs in quantity or price prevent accurate revenue calculations and distort reporting.
```

CRITICAL EVALUATION

<u>What Copilot did well</u>:

- Generated syntactically correct SQL for all 5 audit checks without any errors
- Added meaningful comments explaining not just what each query does but why it matters for data integrity — showing business context awareness
- Used HAVING COUNT(*) > 1 correctly for duplicate detection rather than a less efficient approach
- Correctly used CURRENT_DATE for future-date checking a dynamic function that works regardless of when the query is run

<u>What Copilot missed or got wrong</u>:

- Query 1 detects duplicate invoice_no values but does not check whether the entire row is duplicated — two different transactions can legitimately share context but must have unique invoice numbers. Copilot did not flag this distinction

- Query 3 checks invoice_date > CURRENT_DATE but our dataset stores dates as strings in DD/MM/YYYY format — this query would fail or return incorrect results without first converting the date format using STR_TO_DATE()
- No query was generated to check for inconsistent category or payment_method values — for example, a typo like "Clothng" instead of "Clothing" would pass all 5 checks undetected
- Copilot used FROM retail_sales as the table name but our actual table is named customer_shopping_data it assumed a table name rather than asking for clarification

What I corrected or added:

- Fixed Query 3 to handle the string date format:
  SELECT *
  FROM customer_shopping_data
  WHERE STR_TO_DATE(invoice_date, '%d/%m/%Y') > CURRENT_DATE;
- Added a 6th audit query Copilot missed — checking for unexpected category values:
  SELECT DISTINCT category
  FROM customer_shopping_data
  ORDER BY category;
- Corrected table name from retail_sales to customer_shopping_data throughout

ANALYST VERDICT

Copilot generated 5 clean, well-commented audit queries in seconds — a strong starting point for any data quality review. However, it made a critical assumption about the date format that would have caused Query 3 to fail silently in our MySQL environment, and it missed an entire category of data quality checks (value consistency). Always test AI-generated SQL against your actual data before running in a production environment.

CASE 4: Process Documentation Drafting

Tool: Microsoft Copilot
Objective: Generate a structured process document that a new analyst could follow independently to conduct a monthly data quality audit

BUSINESS PROBLEM

Data operations teams need clear, repeatable process documentation to ensure consistency when team members change or new analysts join. Writing this documentation manually is time-consuming. This use case tests Copilot's ability to convert a technical brief into a structured, professional process guide directly relevant to systems administration and operations roles.

PROMPT USED

I am a data analyst and I need to write a process documentation guide for a new team member explaining how to perform a monthly data quality audit on a retail sales database.

The audit covers these 5 checks:
1. Duplicate invoice numbers
2. Negative or zero values in quantity and price
3. Future-dated transactions
4. Age values outside realistic range
5. NULL values in critical columns

Write a clear, step-by-step process document that a
new analyst could follow independently. Include:
- Purpose of each check
- How to run it
- What to do if issues are found

- How to document findings
Keep it concise and professional.

COPILOT OUTPUT
Monthly Data Quality Audit – Retail Sales Database

Step 1: Check for Duplicate Invoice Numbers

- Purpose: Ensures each transaction is uniquely recorded. Duplicates may inflate revenue or distort customer activity.
- How to Run: Execute a query grouping by invoice_no and flag counts > 1.
- Action if Found: Investigate source system logs; remove or correct duplicates.
- Documentation: Record the number of duplicates, affected invoice numbers, and corrective actions taken.

Step 2: Validate Quantity and Price Values

- Purpose: Prevents invalid transactions that could misstate revenue.
- How to Run: Query for rows where quantity <= 0 or price <= 0.
- Action if Found: Correct obvious entry errors or exclude invalid records from reporting.
- Documentation: Note count of invalid records and resolution approach.

Step 3: Identify Future-Dated Transactions

- Purpose: Ensures transaction dates reflect actual sales activity.
- How to Run: Query for invoice_date > CURRENT_DATE.
- Action if Found: Verify system date settings; adjust or remove erroneous records.
- Documentation: Log number of future-dated entries and corrective measures.

Step 4: Validate Age Range

- Purpose: Maintains realistic customer demographic data for analysis.
- How to Run: Query for age < 15 OR age > 90.
- Action if Found: Investigate source entry; correct if possible, otherwise exclude.
- Documentation: Record count of out-of-range ages and actions taken.

Step 5: Check for NULL Values in Critical Columns

- Purpose: Ensures completeness of key transaction data.
- How to Run: Query for quantity IS NULL OR price IS NULL.
- Action if Found: Attempt to backfill missing values; if not possible, exclude from reporting.
- Documentation: Note the number of NULL records and resolution status.

Final Step – Reporting: Summarise findings in a monthly audit log including issue type, number of records affected, corrective actions, and residual risks. Share with management to ensure transparency and continuous improvement.

CRITICAL EVALUATION

<u>What Copilot did well</u>:

- Produced a clean, consistently structured document; each step follows the same format (Purpose, How to Run, Action if Found, Documentation), making it easy for a new analyst to follow independently.
- Included a Final Step for reporting and escalation not explicitly requested but adds real operational value.
- "Share with management to ensure transparency and continuous improvement" shows awareness of the stakeholder communication dimension of data quality work.

- Language is appropriately professional without being overly technical, suitable for both analyst and management audiences.

What Copilot missed or got wrong:

- Step 3 repeats the same date format error from Case 3: `invoice_date > CURRENT_DATE` will not work correctly on string-formatted dates without STR_TO_DATE() conversion.
- A process document with incorrect syntax would cause a new analyst to fail silently on this check
- No frequency guidance: the document doesn't specify when exactly in the month this audit should run (before month-end close? after data upload?), which is critical for operational scheduling.
- No ownership assignment: a real process document would specify who is responsible for each step and who reviews the final audit log.
- No severity classification: not all issues carry equal risk. A single NULL in price is more critical than an out-of-range age value, but the document treats all findings equally.

What I corrected or added:

- Fixed Step 3 to include correct date conversion syntax for MySQL string-formatted dates
  Added ownership and timing guidance: "This audit should be completed within the first 3 working days of each month, before month-end reporting.
  Primary responsibility: Data Analyst.
  Review responsibility: Senior Analyst or Manager."
- Added severity classification:
  Critical: NULL values in price/quantity, duplicate invoices (impact revenue directly)
  Medium: Future-dated transactions, invalid age values (impact reporting accuracy)

ANALYST VERDICT

Copilot produced a well-structured, immediately usable process document that would give a new analyst a solid starting point. The consistent formatting across all steps is particularly strong. However, the repeated date format error across Cases 3 and 4 highlights a pattern: Copilot does not validate its SQL against your specific database environment and will repeat the same technical assumption across multiple outputs. Human review of any AI-generated SQL is non-negotiable before operational use.
